

Stage 5 实验报告

2022011223 熊泽恩

实验内容

Step 9

为了完成这次实验，我进行了以下工作：

词法分析

为了在函数定义和调用时支持参数列表，所以在 `lex.py` 中添加符号 `t_Comma = ','`。

同时，还需要在 `ply_parser.py` 里定义新的规则：

- `p_program`：现在程序不只由一个 `main` 函数构成，它可以是很多个函数的定义构成。
- `p_function_def`：定义函数的格式。
- `p_parameter_list`：函数声明时的参数列表。
- `p_expression_list`：函数调用时的参数列表。
- `p_function_call`：调用函数的格式。

语法分析

在 `tree.py` 中，我添加了新的 AST 结点类型：`ParameterList`、`Parameter`、`ExpressionList`、`Expression` 和 `Call`，同时，在 `visitor.py` 里添加对应的 Visitor 函数。

语义分析

函数的声明

首先，在此 Step 中，需要判断函数是否被重复定义。所以，我在 `tree.py` 的 `Program` 类中，添加了 `getRedefinedFunc` 成员函数，这样就能在 `namer.py` 的 `Namer.transform` 中，判断是否有函数被重定义。

- 在 `namer.py` 中添加了 `Namer.visitParameter`，作用是在顶层作用域里声明参数符号。
 - 如果顶层作用域里已经有了，那就重复声明，抛出 `DecafDeclConflictError` 异常。
 - 否则，就在顶层作用域里声明此符号。
- 在 `namer.py` 中添加了 `Namer.visitFunction`。
 - 首先，在全局里声明并定义函数符号。
 - 然后，**开一个新的作用域**，遍历参数、访问函数体。
 - 最后，弹出作用域。

函数的调用

- 在 `namer.py` 中添加了 `Namer.visitCall`。
 - 首先，判断是否有此符号，且此符号是否为函数符号，否则，分别抛出 `DecafUndefinedFuncError` 和 `DecafDeclConflictError`。
 - 然后，判断参数列表长度是否正确，否则，抛出 `DecafBadArgCountError`。

- 最后，访问该函数。

生成 TAC

TACFuncEmitter 类

在调用函数时，需要**显式地说明参数**，因此我在 `riscv.py` 中加入了 `DeclParam` 这一指令类型。

TACGen 类

在初始化时，需要添加对参数数量的处理，在初始化 `TACFuncEmitter` 时应该用 `len(astFunc.params)`。

- 在 `TACGen.visitParameter` 中，需要为每个调用时的函数参数分配一个新的临时变量。
- 在 `TACGen.visitCall` 中，需要为调用函数后的返回值分配一个新的临时变量。

生成 RISC-V

栈帧的分配

修改了 `riscvasmemitter.py` 的 `Riscvsubroutineemitter.emitFunc`，在 prologue 和 epilogue 中添加了对栈帧的处理。对应地，需要在初始化的时候，将 `fp` 也考虑进去，将 `nextLocalOffset` 赋值为 `4 * len(Riscv.CalleeSaved) + 8`，而不是原来的 `+4`。

寄存器的分配

1. `localAlloc` 函数：遍历基本块中的每个指令，并为每个指令分配寄存器。这里，需要对 TAC 指令类别进行分类：
 - 对于 `DeclParams`，将前 8 个参数**绑定**到 `a0-a7` 寄存器，其余参数存储到栈上。此时需要记录**每个参数在栈上对应的偏移量**，这也能被看成是一种**绑定**。
 - 对于 `Call`，需要将前 8 个参数**加载**到 `a0-a7` 寄存器，其余参数存储到栈上，同时，还需要保存 caller-saved 的寄存器到栈上。调用结束后，需要恢复这些寄存器，并处理返回值。
 - 对于其他指令，调用 `allocForLoc` 函数进行寄存器分配。
2. **绑定 (bind) 和溢出 (spill)**：一个重要的需要理解的内容是 `bind` 到底是在干什么。`bind` 是指把一个**临时变量**对应到一个**寄存器**的过程，这样在转为 RISC-V 指令时，需要对一个临时变量操作时，可以直接操作它所对应的寄存器。
 - `bind` 方法用于将临时变量绑定到寄存器上。
 - 如果寄存器满了，也就是 Step 13 中所说的“溢出”，则调用 `spill` 函数，并将临时变量从寄存器中解绑。

Stage 5 思考题

Step 9

第 1 题

你更倾向采纳哪一种中间表示中的函数调用指令的设计（一整条函数调用 vs 传参和调用分离）？写一些你认为两种设计方案各自的优劣之处。

具体而言，某个“一整条函数调用”的中间表示大致如下：

```
1 | _T3 = CALL foo(_T2, _T1, _T0)
```

对应的“传参和调用分离”的中间表示类似于：

```
1 PARAM _T2
2 PARAM _T1
3 PARAM _T0
4 _T3 = CALL foo
```

我更倾向于使用“一整条函数调用”的中间表示，在我的代码中，我也采取的是这种方式，主要是因为它在大多数情况下提供了更好的**简洁性**，并且易于理解。当函数调用的参数较少时，这种方式更加直观，不需要通过查看上下文才能发现哪些是参数。

- 一整条函数调用
 - 优点：
 - **简洁**：代码行数更少，阅读起来更加直观。
 - **易于理解**：参数和函数调用紧密相连，可以一眼看出函数调用的上下文。
 - 缺点：
 - **无法处理复杂的参数**：“一整条函数调用”就不方便表示参数变为**变长参数列表**的情况。
- 传参和调用分离
 - 优点：
 - **可以处理复杂的参数**：“传参和调用分离”的方式，可以支持更多复杂的参数，比如变长参数列表。可以看上下文 `PARAM` 的个数来确定参数列表长度。
 - 缺点：
 - **可读性降低**：分离参数和调用会导致代码复杂度增加，使代码不方便理解。
 - **容易出错**：分离参数还容易引入错误，比如**参数顺序**可能会变得混乱。

第 2 题

为何 RISC-V 标准调用约定中要引入 callee-saved 和 caller-saved 两类寄存器，而不是要求所有寄存器完全由 caller/callee 中的一方保存？为何保存返回地址的 `ra` 寄存器是 caller-saved 寄存器？

我认为，引入 callee-saved 和 caller-saved 两类寄存器，主要是为了**提升性能**。

- 如果 caller 在调用多个函数时需要保持某些数据不变，可以把这些数据放在 callee-saved 寄存器中，这样 caller 就不必在每次调用前后保存和恢复这些数据。
- 对应地，对于临时数据，就可以使用 caller-saved 寄存器，因为 callee 可以在调用前后自由使用这些寄存器，而不必 spill 对应的寄存器到内存。

对于 `ra` 而言，`ra` 中存放的是当前函数（记为 A）作为 callee，返回它自己的 caller（记为 F）时，所需要用的返回地址。而 A 作为 caller，在调用 callee（记为 B）的过程中，会修改 `ra`，变为从 B 返回到 A 时所需要用的返回地址。如果 `ra` 是 callee-saved 的，那么这个 B 到 A 的返回地址会覆盖 A 到 F 的返回地址，导致错误。因此，`ra` 是 caller-saved 寄存器。

Honor Code

我在回答 Step 9 思考题第 2 题时，参考了以下资料：

1. [assembly - Why \\$ra is Caller Saved in RISC-V - Stack Overflow](#)
2. [计算机组成-寄存器保存 | 钟鼓楼 \(thysrael.github.io\)](#)